

Progress Report

Day 22 — Kanban Filters and Optimistic Updates

Task Management MERN Stack Platform

Shafin Nigamana | 10 June 2026

Tasks Completed

Filter Chips

- Added quick filter chips to the Kanban board.

Available filters:

- All
 - My Tasks
 - High Priority
 - Overdue
- Implemented client-side filtering for instant task visibility updates.

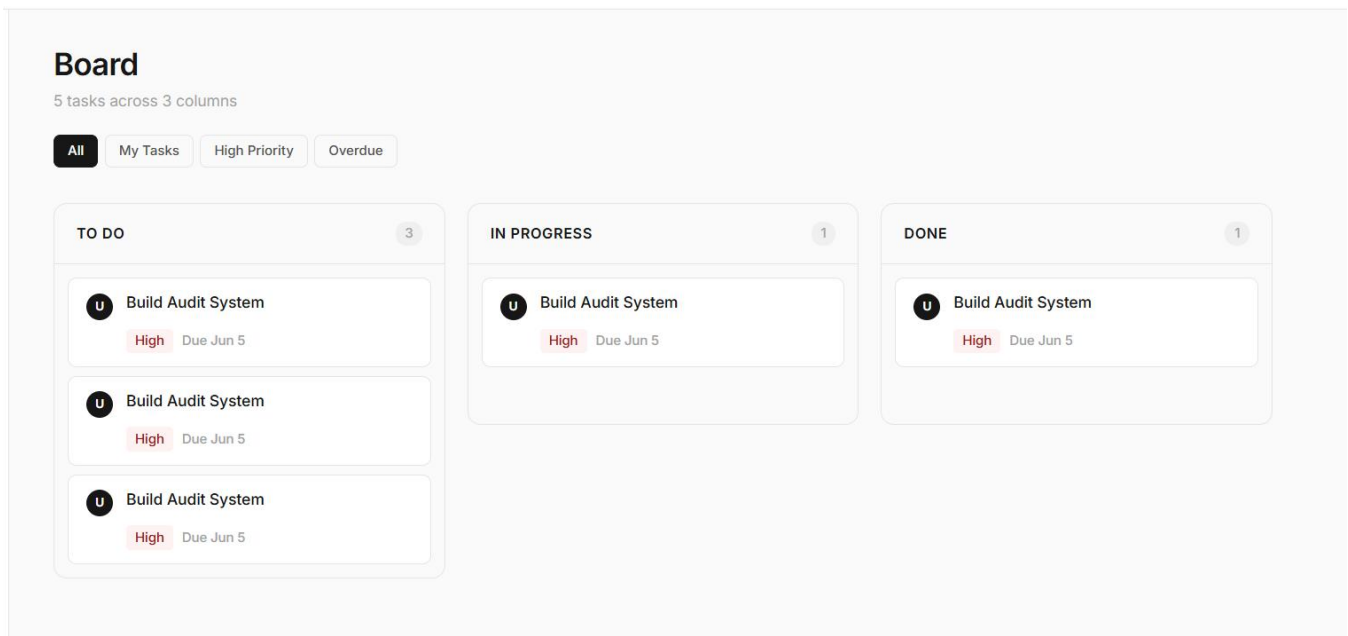


Figure 2.1 — Kanban board displaying task filter chips.

My Tasks Filter

- Added filtering based on the logged-in user.
- Displayed only tasks assigned to the current user.
- Integrated user information from AuthContext.

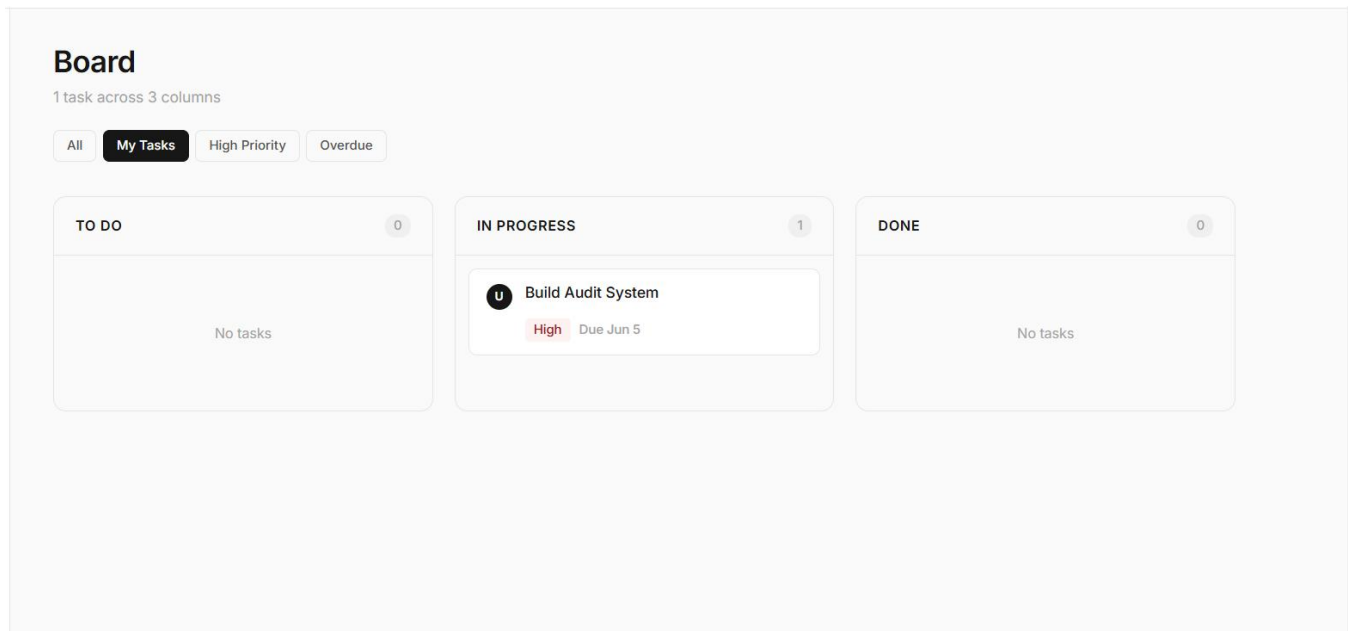


Figure 2.2 — My Tasks filter displaying tasks assigned to the logged-in user.

High Priority and Overdue Filters

- Added filtering for high-priority tasks.
- Added filtering for overdue tasks based on due-date validation.
- Updated board counts dynamically when filters are applied.

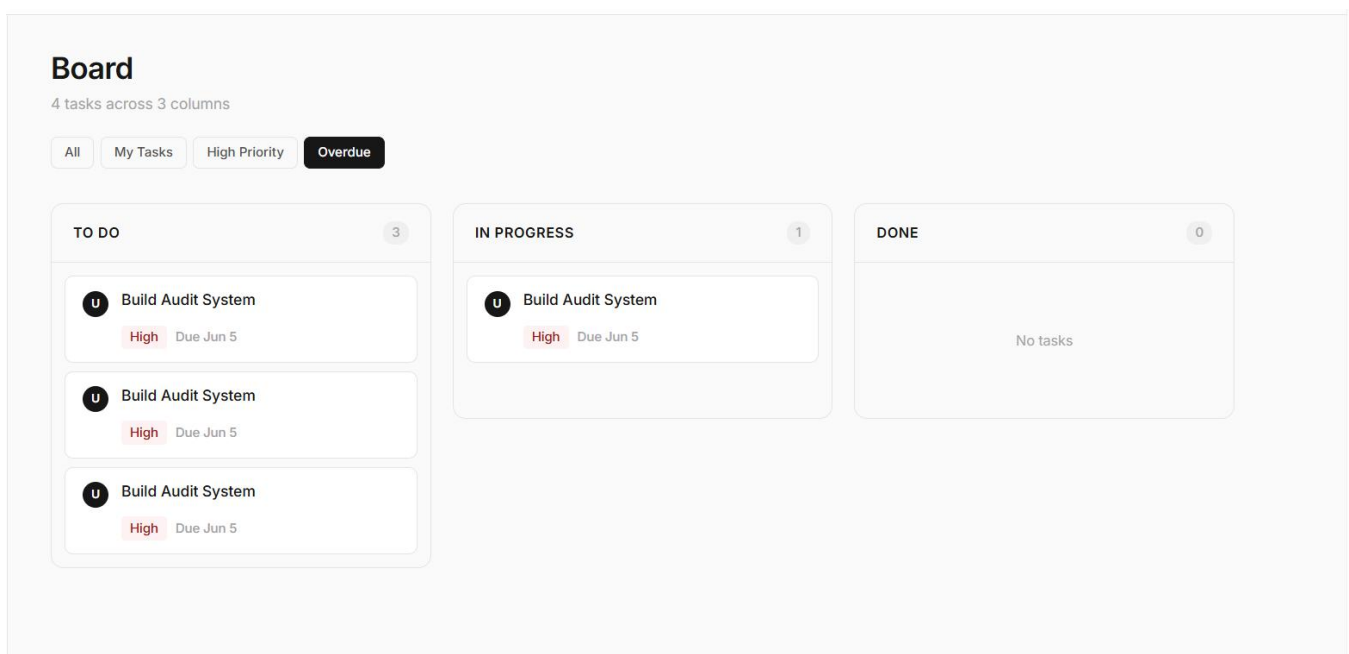


Figure 2.3 — High Priority and Overdue filters applied to the Kanban board.

Optimistic UI Updates

- Improved drag-and-drop responsiveness using optimistic updates.
- Tasks move immediately within the interface before API confirmation is received.
- Maintained synchronization with backend task status updates.

```
async function handleDragEnd(event) {
  isDragging.current = false;
  setActiveTask(null);
  setOverColumnId(null);

  const { active, over } = event;
  if (!over) return;

  const taskId = active.id;
  const newStatus = getTargetColumn(over);

  if (!newStatus) return;

  const currentStatus = findColumnForTask(taskId);
  if (currentStatus === newStatus) return;

  // Optimistic update – move card immediately
  setTasks((prev) =>
    prev.map((t) =>
      t._id === taskId ? { ...t, status: newStatus } : t
    )
  );
}
```

Figure 2.4 — Optimistic task movement implemented during drag-and-drop interactions.

Rollback Handling

- Added rollback functionality for failed task status updates.
- Restored task status automatically when API requests fail.
- Prevented inconsistent Kanban board states during update errors.

```
if (!newStatus) return;

const currentStatus = findColumnForTask(taskId);
if (currentStatus === newStatus) return;

// Optimistic update – move card immediately
setTasks((prev) =>
  prev.map((t) =>
    t._id === taskId ? { ...t, status: newStatus } : t
  )
);

// Persist to backend via PATCH
try {
  await updateTaskStatus(taskId, newStatus);
} catch (err) {
  // Rollback – restore previous status on failure
  setTasks((prev) =>
    prev.map((t) =>
      t._id === taskId ? { ...t, status: currentStatus } : t
    )
  );
}
```

Figure 2.5 — Rollback mechanism implemented for failed task update requests.